MNIST 손글씨 인식 웹 서비스 보고서

perweekdev@gmail.com

1. 프로젝트 개요

1.1. 미션 목적 및 요구사항

- 웹에서 사용자가 숫자를 그리면 실시간으로 예측하는 Streamlit 기반 AI 서비스를 구현하는 과제이다.
- ONNX 모델 저장소의 MNIST ONNX 모델을 활용하여 손글씨 숫자 분류 기능을 구현하였다.
- 입력 UI, 전처리 시각화, 결과 저장, Docker 배포까지 포함한 서비스 개발을 목표로 하였다.
- **주요 요구사항**
  - ONNX 모델 로딩 및 캐싱
  - 드로잉 입력 캔버스 구현 및 전처리 이미지 표시
  - 0~9 예측 확률 시각화
  - 저장 기능 및 저장 내역 제공
  - Docker 이미지 빌드 및 Docker Hub 배포

1.2. 개발 범위 및 산출물

- **개발 범위**: Streamlit 앱, ONNX Runtime 연동, 전처리 파이프라인, 저장 기능, Docker 배포
- **주요 산출물**
  - `소스코드 ZIP(Dockerfile, docker-compose.yml 포함)`
  - `Docker Hub URL`
  - `보고서 PDF`

2. 시스템 설계 및 구현

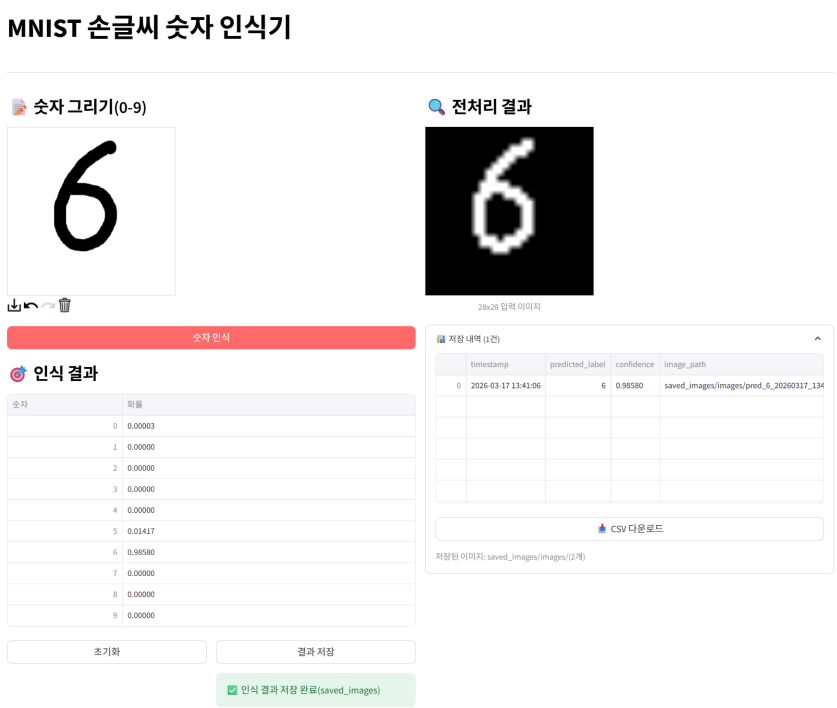
2.1. 전체 아키텍처

- Streamlit 단일 앱 구조로 UI와 처리 로직을 함께 구성하였다.
- 모델은 ONNX Runtime 세션으로 로딩하고, Streamlit 캐시를 활용해 재사용하도록 설계하였다.
- **서비스 플로우**
  1. 사용자가 캔버스에 숫자를 그림
  2. 캔버스 이미지 수집, 28×28 형태로 전처리
  3. ONNX 모델 추론
  4. 예측 확률 및 대표 숫자 출력
  5. PNG 및 CSV 저장

2.2. 기술 스택 및 화면 구성

구분	기술	역할
웹 UI	Streamlit	전체 앱 구성
입력	streamlit-drawable-canvas	손글씨 입력
추론	ONNX Runtime	모델 실행
모델	MNIST ONNX	숫자 분류
전처리	Pillow, NumPy	이미지 처리
데이터 저장	pandas	CSV 생성
배포	Docker, Docker Compose	컨테이너 실행

화면 구성



- 왼쪽: 입력 캔버스, 예측 버튼, 예측 결과
- 오른쪽: 전처리 이미지, 저장 내역, CSV 다운로드

2.3. 모델 처리 및 Streamlit 구현

- 사용 모델: MNIST ONNX 모델 (`mnist-8.onnx` 기준)
- 입력 형태: `(1,1,28,28)` float32 텐서
- 모델 출력: `(1,10)` 로짓 벡터, softmax 적용 후 확률 변환
- 구현 분리
  - `main.py`: UI 및 전체 흐름, `model_utils.py`: 모델 로딩, 추론, `image_utils.py`: 이미지 전처리

3. 핵심 기능 개발

3.1. 입력 캔버스 및 전처리 시각화

- `st_canvas()`를 활용해 자유롭게 숫자를 입력할 수 있도록 구현하였다.
- 주요 설정
  - 흰색 배경, 검은색 선(선 두께 20)
  - 캔버스 크기 280×280
- 전처리 단계
  - RGBA to PIL 이미지 변환 및 그레이스케일 변환
  - 28×28 리사이즈 및 Otsu 이진화
  - 색 반전(흰 배경 검정 글씨 → 검정 배경 흰 글씨)
  - 정규화 및 텐서 변환
- 전처리 결과 이미지를 우측 패널에 함께 보여주어 입력과 모델 입력의 차이를 확인할 수 있도록 하였다.

3.2. 예측 확률 시각화 및 저장소 관리

- ONNX Runtime 추론 결과에 softmax를 적용하여 0~9 확률을 계산하였다.
- 예측 결과는 대표 레이블과 클래스별 확률 테이블로 표시하였다.
- 확률 값은 소수점 다섯 자리까지 출력하도록 구현하였다.
- 저장 기능
  - "결과 저장" 클릭 시 즉시 저장
  - 전처리 이미지는 PNG로 저장
  - 예측 레이블, confidence, `prob_0~prob_9`는 CSV에 저장
- 저장 경로
  - 이미지: `saved_images/images/`
  - CSV: `saved_images/predictions.csv`
- CSV에는 이미지 자체는 없지만 `image_path`를 함께 저장해 파일과 연결되도록 하였다.

3.3. 사용자 경험 개선 및 예외 처리

- 초기화 버튼 동작 개선
  - 세션 상태 초기화만으로는 캔버스가 지워지지 않아 `canvas_reset_key` 방식으로 수정
  - 차트와 캔버스를 동시에 초기화하도록 구현
- UI 단순화
  - 저장 내역은 간단 보기 기준으로 고정
  - 다운로드 버튼만 유지(로컬 저장은 자동으로 수행되므로 삭제)
- 예외 처리
  - 전처리 실패 시 오류 메시지 출력

4. Docker 배포 및 검증

4.1. Dockerfile 설계 및 로컬 테스트

- Streamlit Docker 배포 가이드를 참고해 Dockerfile을 작성하였다.
- 베이스 이미지는 `python:3.10-slim`을 사용하였다.
- Dockerfile 주요 단계
  - 시스템 패키지 설치, requirements 설치
  - 프로젝트 복사, 저장 폴더 생성
  - Streamlit 실행
- 로컬 테스트
  - `docker-compose up --build`
  - `http://localhost:8501` 접속
  - 앱 기능 및 저장 기능 정상 동작 확인

4.2. Docker Hub 업로드 및 실행 검증

- Docker Hub 로그인 후 이미지 태그와 push를 수행하였다.
- 업로드 이미지
  - `perweekdev/mnist-handwriting:latest`
- 실행 명령어

```
docker pull perweekdev/mnist-handwriting:latest
docker run -p 8501:8501 perweekdev/mnist-handwriting:latest
```

- 로컬 컨테이너 실행 및 Docker Hub 업로드까지 완료하여 재현 가능한 상태를 확보하였다.

5. 실험 평가 및 결론

5.1. 테스트 결과 및 성능 분석

- 테스트 항목
  - 0~9 숫자 입력
  - 저장 기능 및 CSV 생성 확인
  - Docker 환경 실행 확인
- 결과
  - 기본 숫자 인식은 전반적으로 안정적
  - 1, 4, 7은 비교적 정확도가 높음
  - 3, 8처럼 유사한 숫자에서 일부 오분류 발생

- 성능
  - 모델의 예측은 체감상 요청시 즉시 수행됨
  - 캐시 적용 후 반복 추론 시 안정성 향상
  - 저장 기능도 예측 직후 즉시 반영됨

5.2. 프로젝트 요약 및 향후 개선 방향

- 본 프로젝트를 통해 손글씨 숫자 인식 웹 서비스를 완성하였다.
- 구현 완료 항목
  - Streamlit 기반 손글씨 입력 UI, 전처리 시각화
  - MNIST ONNX 추론
  - PNG/CSV 저장 기능
  - Docker 배포 및 Docker Hub 업로드
- 향후 개선 방향
  - 숫자 중심 정렬 기반 전처리 추가
  - 모바일 입력 최적화
  - 다중 모델 비교 기능 확장

부록

Docker Hub URL: https://hub.docker.com/r/perweekdev/mnist-handwriting

실행 명령어

```
docker pull perweekdev/mnist-handwriting:latest
docker run -p 8501:8501 perweekdev/mnist-handwriting:latest
```